Instructions for Lab Submission

1. **File Naming and Submission:**
   a. Submit your solution as a **ZIP file** named `lab<number>_<ERP>.zip`, where `<ERP>` is your ERP number. For example, if your ERP number is 12345, the file should be named `lab1_12345.zip`.
   b. Ensure that the ZIP file contains:
      i. C++ source files named as `task<number>.cpp` (e.g., `task1.cpp`).
      ii. No unnecessary files (such as executables, temporary files, or editor configs).
   c. Do not submit **TEXT(.txt)** or **EXE(.exe)** files.
2. **Code Organization:**
   a. Write each task in its own `.cpp` file (as specified above).
   b. Ensure that your code is modular and logically separated into functions when specified.
3. **Code Neatness:**
   a. Ensure your code is **well-formatted** and easy to read. Use proper **indentation** and **meaningful variable names**.
   b. Include appropriate **comments** to explain the logic where necessary (especially for functions and complex sections of the code).
   c. **Add clear explanations where required (or specified)** to make the logic of your program easy to follow.
4. **Code Compilation and Functionality:**
   a. Ensure your code compiles without errors or warnings. You should be able to compile and run your code on any system with a standard C++ compiler (e.g., GCC, Clang, MSVC).
   b. Double-check that the functionality of the program meets the requirements outlined in the task.
5. **Additional Instructions:**
   a. Do not use any libraries or features that have not been covered in the course material, unless specifically instructed.
   b. Make sure your code adheres to the principles of good software development (e.g., modular functions, minimal repetition, and no hardcoding).
   c. If you encounter any issues or require clarifications, reach out before the submission deadline.
6. **Deadline:**
   a. Ensure that you submit your lab on time, before the deadline. Late submissions will not be accepted
   b. Double-check that the zip file is correctly named and contains all necessary files before submitting.
7. **Plagiarism Policy:**
   a. The work submitted must be entirely your own. Any code or content copied from others (including online sources, classmates or AI) will result in an automatic zero for the task.

Lecture Review

1. Functions

A **function** is a named block of code that performs a specific task. Functions help divide a program into smaller, reusable parts.

For example:

```
int multiply(int n, int m) {
    return n * m;
}
```

A function can be called whenever its task is needed:

```
int result = multiply(5, 4);
cout << result;
```

Functions improve code reusability, organization, readability, and debugging.

2. Function Declaration and Definition

A **function declaration** tells the compiler the function's name, return type, and parameters.

```
int add(int a, int b);
```

A **function definition** contains the statements that perform the task.

```
int add(int a, int b) {
    return a + b;
}
```

A complete example:

```
#include <iostream>
using namespace std;

int add(int a, int b);

int main() {
    cout << add(5, 3);
    return 0;
}

int add(int a, int b) {
    return a + b;
}
```

Output:

```
8
```

A function must be declared before it is called if its definition appears later in the program.

3. Parameters and Arguments

Parameters are variables listed in a function's definition. **Arguments** are the actual values passed to the function when it is called.

```
int add(int a, int b) {
    return a + b;
}
```

```
int main() {
    int result = add(5, 3);
    cout << result;
    return 0;
}
```

In this example, `a` and `b` are parameters, while `5` and `3` are arguments.

4. Pass-by-Value and Pass-by-Reference

In **pass-by-value**, a copy of the argument is passed to the function. Changes made to the parameter do not affect the original variable.

```
void change(int n) {
    n = 100;
}

int main() {
    int number = 10;
    change(number);
    cout << number;  // 10
    return 0;
}
```

In **pass-by-reference**, the parameter refers to the original variable. Changes made to the parameter affect the original variable.

Use the reference operator `&` in the parameter declaration.

```
void change(int &n) {
    n = 100;
}

int main() {
    int number = 10;
    change(number);
    cout << number;  // 100
    return 0;
}
```

Comparison

Feature	Pass-by-Value	Pass-by-Reference
Parameter	Receives a copy	Refers to original variable
Changes original variable?	No	Yes
Parameter syntax	`int n`	`int &n`

5. Return Types and the `return` Statement

The return type specifies the type of value a function gives back.

Common return types include:

- `int`: returns an integer.
- `double`: returns a decimal number.
- `char`: returns a character.
- `bool`: returns `true` or `false`.
- `void`: does not return a value.

Example of a value-returning function:

```
int multiply(int a, int b) {
    return a * b;
}
```

Example of a void function:

```
void display() {
    cout << "Welcome!";
}
```

The `return` statement sends a value back to the calling code and ends the function's execution.

6. Default Parameters

A **default parameter** is a parameter that has a predefined value. If the caller does not provide an argument for that parameter, the default value is used.

```
double calculatePrice(double price, double discount = 5.0) {
    return price - (price * discount / 100);
}
```

The function can be called in two ways:

```
cout << calculatePrice(1000);       // Uses 5% discount
cout << calculatePrice(1000, 10);   // Uses 10% discount
```

Important rules:

- Default values are specified in the function declaration or definition, not repeatedly in both.
- Parameters with default values must appear after parameters without default values.

7. Local and Global Variables

A **local variable** is declared inside a function or block and is generally accessible only within that scope.

```
void display() {
    int number = 10;
    cout << number;
}
```

A **global variable** is declared outside all functions and can be accessed by functions within its scope.

```
int number = 10;

void display() {
    cout << number;
}
```

Local variables are useful for temporary calculations. Global variables should be used carefully because changes to them can affect multiple parts of a program.

8. Passing Arrays to Functions

Arrays can be passed to functions so that the function can process or modify their elements.

For a one-dimensional C-style array, the function commonly receives the array and its size as separate parameters.

```
void displayArray(int arr[], int size) {
    for (int i = 0; i < size; i++) {
        cout << arr[i] << " ";
    }
}
```

Calling the function:

```
int numbers[5] = {2, 4, 6, 8, 10};

displayArray(numbers, 5);
```

A function can also modify the original array elements:

```
void increaseArray(int arr[], int size) {
    for (int i = 0; i < size; i++) {
        arr[i]++;
    }
}
```

9. The `std::array` Container

The `std::array` container represents a fixed-size array. Unlike a normal C-style array, it is part of the C++ Standard Library and provides useful member functions.

It is available through the `array` header.

```
#include <iostream>
#include <array>
using namespace std;

int main() {
    array<int, 5> numbers = {2, 4, 6, 8, 10};

    for (int i = 0; i < numbers.size(); i++) {
        cout << numbers[i] << " ";
    }

    return 0;
}
```

The size of an `std::array` is specified when it is declared and cannot be changed later.

Passing an std::array to a function

An `std::array` can be passed by value or by reference.

Passing by value creates a copy:

```
void displayArray(array<int, 5> numbers) {
    for (int i = 0; i < numbers.size(); i++) {
        cout << numbers[i] << " ";
    }
}
```

Passing by reference allows the function to modify the original array:

```
void increaseArray(array<int, 5> &numbers) {
    for (int i = 0; i < numbers.size(); i++) {
        numbers[i]++;
    }
}
```

Returning an std::array from a function

A function can return an entire `std::array`.

```
array<int, 3> createArray() {
    array<int, 3> numbers = {10, 20, 30};
    return numbers;
}
```

Calling the function:

```
array<int, 3> result = createArray();
```

10. Vectors

A `vector` is a dynamic array that can grow or shrink in size. It is available through the `vector` header.

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<int> numbers;

    numbers.push_back(10);
    numbers.push_back(20);
    numbers.push_back(30);

    for (int i = 0; i < numbers.size(); i++) {
        cout << numbers[i] << " ";
    }

    return 0;
}
```

Some useful vector operations:

- `push_back(value)`: Adds an element at the end.
- `pop_back()`: Removes the last element.
- `size()`: Returns the number of elements.
- `empty()`: Checks whether the vector is empty.

Passing a vector to a function

A vector can be passed by value or by reference.

Passing by value creates a copy:

```
void displayVector(vector<int> numbers) {
    for (int i = 0; i < numbers.size(); i++) {
        cout << numbers[i] << " ";
    }
}
```

Passing by reference allows the function to modify the original vector:

```
void doubleValues(vector<int> &numbers) {
    for (int i = 0; i < numbers.size(); i++) {
        numbers[i] *= 2;
    }
}
```

Returning a vector from a function

A function can return a vector containing the required values.

```
vector<int> createVector() {
    vector<int> numbers;

    numbers.push_back(5);
    numbers.push_back(10);
    numbers.push_back(15);

    return numbers;
}
```

Calling the function:

```
vector<int> result = createVector();
```

11. Two-Dimensional Vectors

A two-dimensional vector stores multiple vectors. It can be used to represent rows and columns, similar to a matrix.

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<vector<int> > matrix = {
        {1, 2, 3},
        {4, 5, 6},
        {7, 8, 9}
    };

    for (int i = 0; i < matrix.size(); i++) {
        for (int j = 0; j < matrix[i].size(); j++) {
            cout << matrix[i][j] << " ";
        }

        cout << endl;
    }

    return 0;
}
```

The expression `matrix[i]` refers to an entire row, while `matrix[i][j]` refers to an individual element.

A row can be passed to a function as a vector.

```
void displayRow(vector<int> row) {
    for (int i = 0; i < row.size(); i++) {
        cout << row[i] << " ";
    }
}
```

A row can also be passed by reference when the function needs to modify the original row.

```
void modifyRow(vector<int> &row) {
    for (int i = 0; i < row.size(); i++) {
        row[i] *= 2;
    }
}
```

For example, the following call passes the second row of a matrix to the function:

```
modifyRow(matrix[1]);
```

12. Recursion

Recursion occurs when a function calls itself to solve a smaller version of the same problem.

A recursive function must have a **base case** that stops further recursive calls. Each recursive call should move the problem closer to this base case.

Example 1: Sum of Natural Numbers

The following function calculates the sum of all positive integers from 1 to n.

```
int sum(int n) {
    if (n == 0) {
        return 0;
    }

    return n + sum(n - 1);
}
```

Example 2: Power Calculation

The following function calculates the value of a number raised to a non-negative integer power.

```
int power(int base, int exponent) {
    if (exponent == 0) {
        return 1;
    }

    return base * power(base, exponent - 1);
}
```

13. File Handling

C++ provides file handling through the `fstream` library.

The following classes are commonly used:

- `ifstream`: Used to read data from a file.
- `ofstream`: Used to write data to a file.
- `fstream`: Used for both reading and writing.

Reading from a file

To read data from a file, create an `ifstream` object and open the required file.

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
    ifstream file("numbers.txt");

    int number;

    if (!file) {
        cout << "Unable to open file." << endl;
        return 1;
    }

    while (file >> number) {
        cout << number << endl;
    }

    file.close();

    return 0;
```

```
}
```

The expression `file >> number` reads the next available integer from the file. The loop continues until no more integers can be read.

Passing a filename to a function

A filename can be passed to a function so that the function can open and process the file.

```
void readFile(string filename) {
    ifstream file(filename);

    if (!file) {
        cout << "Unable to open file." << endl;
        return;
    }

    int number;

    while (file >> number) {
        cout << number << " ";
    }

    file.close();
}
```

The function can receive a filename as a parameter and perform file operations without requiring the file to be opened in `main()`.

Writing to a file

An `ofstream` object can be used to write data to a file.

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
    ofstream file("output.txt");

    if (!file) {
        cout << "Unable to create file.";
        return 1;
    }

    file << 10 << " " << 20 << " " << 30;

    file.close();

    return 0;
}
```

14. Important Reminders

- A function is a reusable block of code that performs a specific task.
- Parameters are variables in a function definition; arguments are values passed during a function call.
- Pass-by-value creates a copy, while pass-by-reference allows a function to modify the original variable.
- A function can return a value, including an `std::array` or a vector.
- `std::array` has a fixed size, while a vector can change its size.
- Use `array` and `vector` headers when working with these containers.
- The `size()` function returns the number of elements in an array or vector.
- A two-dimensional vector can represent rows and columns.
- Passing a row of a two-dimensional vector by reference allows a function to modify that row in the original matrix.

- Recursion requires a base case to stop recursive calls.
- Each recursive call should move towards a base case.
- Selection Sort repeatedly selects the smallest remaining element and places it in its correct position.
- `ifstream` is used to read data from files.
- `ofstream` is used to write data to files.
- Always check whether a file opened successfully before attempting to read from or write to it.
- Close files after completing file operations.

Lab Tasks

Task 1: Student Scores File Log System

Write a C++ program `task1.cpp` that performs basic file input/output to store and retrieve student assessment scores.

Problem Description

Prompt the user to enter 5 integer exam scores. Use an `ofstream` object to write these scores line-by-line into a text file named `scores.txt`. Then, use an `ifstream` object to open the file, read the scores back, print them to the console, and compute their overall average.

Requirements

- Include the `<fstream>` library.
- Verify that files open successfully before performing read or write operations.
- Close all file streams properly upon completion.
- Do not calculate the total sum of scores and divide it by the number of scores. Instead, calculate the running average while reading the scores from the file.
- Update the average at each iteration.

Example Output

```
Scores successfully written to scores.txt.
Reading scores from file: 85 90 78 92 88
Average Score: 86.6
```

Task 2: Recursive Countdown & Factorial Display

Write a C++ program `task2.cpp` that implements two simple recursive functions: a countdown printer and a recursive factorial calculator.

Problem Description

Write a recursive function `void countdown(int n)` that prints numbers from `n` down to `1` followed by `"Liftoff!"`, and a recursive function `long long factorial(int n)` that computes $n!$. Prompt the user for a number, call both recursive functions, and display the results.

Requirements

- Implement both functions using direct recursion with proper base cases.
- Handle input and output in `main()`.

Example Output

```
Enter a positive integer: 5
Countdown: 5 4 3 2 1 Liftoff!
Factorial of 5 is: 120
```

Task 3: Number Sequence Generator

Write a C++ program `task3.cpp` that passes a `std::vector<int>` to a function to generate and display a sequence for each of its elements.

Problem Description

Write a function `void generateSequence(const vector<int>& numbers)` that takes a vector of positive integers and generates a sequence for each element.

For each number, apply the following rules repeatedly until the value becomes 1:

- If the number is even, divide it by 2.
- If the number is odd, multiply it by 3 and add 1.

Display each generated sequence on a separate line.

In `main()`, create a vector, populate it with 5 positive integers from the user, and pass it to the function.

Requirements

- Pass the vector by constant reference (`const vector<int>&`).
- Use a standard `for` loop inside the function to traverse the vector elements.
- Use only a `for` loop inside the function to generate each sequence. Do not use `while` or `do-while` loops.
- Display the original number and all subsequent values, including 1.

Example Output

```
Enter 5 numbers: 3 5 7 8 10

Sequence for 3: 3 10 5 16 8 4 2 1
Sequence for 5: 5 16 8 4 2 1
Sequence for 7: 7 22 11 34 17 52 26 13 40 20 10 5 16 8 4 2 1
Sequence for 8: 8 4 2 1
Sequence for 10: 10 5 16 8 4 2 1
```

Task 4: 2D Matrix Row Sum Calculator

Write a C++ program `task4.cpp` that takes a fixed-size 2D array representing a matrix, passes it to a function, and computes the sum of elements for each individual row.

Problem Description

Implement a function `void rowSums(int mat[][3], int rows, int cols)` that calculates the sum of each row in a given 3x3 2D array and prints the calculated total for each row. Call this function from `main()`.

Requirements

- Pass the 2D array to the function specifying column dimensions (e.g., `int mat[][3]`).
- Use nested loops to iterate through rows and columns to compute each row's total.
- Print each row's sum inside the function or store and display them in `main()`.

Example Output

```
Row 1 Sum: 15
Row 2 Sum: 18
Row 3 Sum: 21
```

Task 5: Recursive Digit Counter (`count7`)

Write a C++ program `task5.cpp` that uses recursion to count how many times the digit `7` appears in a non-negative integer.

Problem Description

Write a recursive function with the signature `int count7(int n)` that returns the count of occurrences of the digit `7` in `n`. For example, `count7(717)` should yield `2`.

Requirements

- Solve the problem using recursion (do not use loops).
- Use modulus (`% 10`) to isolate the rightmost digit and division (`/ 10`) to reduce the number.

Example Output

```
Enter a non-negative integer: 71707
Occurrences of digit 7: 3
```